

Parke Transformation Module Documentation

1. Park Transformation Module (Park.h / Park.c)

1.1. Overview

This module implements the **Park transformation** and its inverse, which constitute the second stage of the two-step reference frame conversion used in vector control of three-phase power-electronics systems. Whereas the Clarke transformation (described in the preceding section) maps three-phase ABC quantities into the stationary orthogonal $\alpha\text{-}\beta\text{-}0$ frame, the Park transformation further rotates the $\alpha\text{-}\beta$ plane by an electrical angle θ , yielding the **synchronously rotating** $dq0$ reference frame.

In steady state, sinusoidal $\alpha\text{-}\beta$ quantities become DC quantities in the dq frame, which greatly simplifies controller design: PI regulators with zero steady-state error can be applied directly to the d - and q -axis components.

1.2. Mathematical Convention

The Park transformation rotates the stationary $\alpha\text{-}\beta$ frame by the electrical angle θ to obtain the rotating dq frame. The rotation matrix applied is:

$$\mathbf{T}_P(\theta) = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \quad (1)$$

yielding the forward transformation:

$$d = \alpha \cos \theta + \beta \sin \theta \quad (2)$$

$$q = -\alpha \sin \theta + \beta \cos \theta \quad (3)$$

$$0 = 0_{\alpha\beta} \quad (4)$$

The inverse transformation applies the transpose of $\mathbf{T}_P$, which equals its inverse since the matrix is orthogonal ($\mathbf{T}_P^{-1} = \mathbf{T}_P^\top$):

$$\alpha = d \cos \theta - q \sin \theta \quad (5)$$

$$\beta = d \sin \theta + q \cos \theta \quad (6)$$

$$0 = 0_{dq} \quad (7)$$

Axis alignment convention: The implementation follows the standard convention in which the d -axis is aligned with the reference phasor—in this case, the grid voltage vector $\vec{v}_g$. Under this alignment, the grid voltage satisfies $v_g^q \approx 0$ in steady state, so that the active power is controlled exclusively through i_g^d and the reactive power through i_g^q . This convention is consistent with the control design described in the DC-link and current controller sections.

Remark on sign convention: Some references define the q -axis with an opposite sign (i.e., $q = \alpha \sin \theta - \beta \cos \theta$). The convention adopted here places the q -axis leading the d -axis by 90 in the counter-clockwise direction, which is the standard in IEEE and IEC power-systems literature.

1.3. Data Structures

`Park.h` defines one new structure and reuses one from the Clarke module:

- `FORM_PARK`: Stores the d , q , and zero-sequence components, each declared as `double`. This is the output type of the forward transformation and the input type of the inverse.
- `FORM_CLARKE`: Defined in `Clarke.h` and directly included via `#include <Clarke.h>` at the top of `Park.h`. It is reused as the output type of the inverse Park transformation, enforcing structural consistency across the transformation chain.

1.4. Implemented Functions

1.4.1. Transform_Clarke_to_Park

```
void Transform_Clarke_to_Park(FORM_PARK *park, float alpha, float beta,
                             float zero, float theta);
```

Computes equations (2)–(4) and writes the result into the structure pointed to by `park`. The angle `theta` must be provided in **radians** and is consumed through the standard C library functions `cos()` and `sin()` from `<math.h>`, which operate on `double` arguments. a widening conversion occurs prior to the trigonometric evaluation.

The zero-sequence component is passed through unchanged, as it is invariant under the rotational transformation.

1.4.2. Transform_Park_to_Clarke

```
void Transform_Park_to_Clarke(FORM_CLARKE *clarke, float d, float q,
                             float zero, float theta);
```

Implements the inverse transformation given by equations (5)–(7). The output is written into a `FORM_CLARKE` structure, making it directly compatible with the input expected by `Transform_Clarke_to_ABC` from the Clarke module.

Both functions are stateless and reentrant, containing no internal static variables or shared mutable data.

1. **Feedback path (sensing):** The α - β signals produced by `Transform_ABC_to_Clarke` are fed into `Transform_Clarke_to_Park` using the angle θ supplied by the PLL. The resulting i_g^d and i_g^q components are then consumed by the inner current PI controllers.
2. **Modulation path (output):** The voltage references v_d^* and v_q^* produced by the current controllers and the decoupling feedforward are converted back to the stationary frame via `Transform_Park_to_Clarke`, before being further processed by `Transform_Clarke_to_ABC` to recover the per-phase modulation signals.

1.5. Dependencies

1.5.1. Intra-project Dependencies

- `Clarke.h`: Explicitly included in `Park.h` via a direct `#include` directive. This dependency is mandatory, as the `FORM_CLARKE` type—defined therein—is used as the output type of `Transform_Park_to_Clarke`. Any compilation unit that includes `Park.h` will therefore also transitively include `Clarke.h`.

2. References

This module was provided by the PEC Lab and it needs to be referenced before sharing this project